

OPTIMISATION BDD · SHOPFLOW · SUP DE VINCI M2 · ATELIER 10

Dossier final : le chargement groupé de GET /commandes, et le prix d'un cache

Question du dossier. Une page de 20 commandes paraît simple mais devient lente quand plusieurs clients arrivent : d'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ? Chaque chiffre renvoie à une preuve **P-xx** (fiches p. D-7), à un fichier de mesures (atelier3/ à atelier9/) ou à une page de PRINCIPAL.pdf (« PRINCIPAL p. N »). La présentation de 5 minutes est dans Presentation_V1_structure.pdf et Presentation_V2_histoire.pdf.

Inventaire des preuves déjà produites (ateliers 1 à 8, Jour 5)

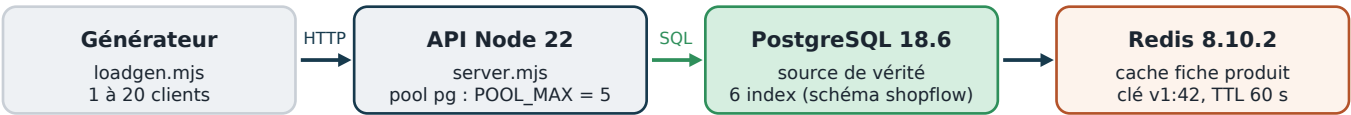
Famille	Résultat mesuré (preuve déjà réalisée)	Où	Rôle ici
Plans et index	Atelier 3 : index composé (client_id, created_at DESC, id DESC) : ×2,5 sur la requête de l'API, ×91 pour un client à 20 100 commandes, WAL +35 % ; retenu, migration écrite et non appliquée au laboratoire. Atelier 4 : index partiel 10,1 → 0,070 ms (×144,6, 1 680 → 3 buffers) mais inutile hors de son prédicat	PRINCIPAL p. 7-15	contexte
Réécriture et tuning	Atelier 2 : EXISTS 11,3 ms contre jointure 27,6 ms (la jointure répète le client 42 cent fois). Agrégation mensuelle sur copie : 84,6 → 50,2 ms (work_mem 32 MB) ou 45,5 ms (statistique), même résultat (md5)	PRINCIPAL p. 1-6	contexte
Migration	Atelier 4 bis : NOT NULL direct = 3 324,1 ms sous verrou exclusif, migration progressive = 46,3 ms ; remplissage 2,7 fois plus long ; retour arrière vérifié	PRINCIPAL p. 16-18	contexte
API	Atelier 7 : N+1 21 → 2 requêtes ; curseur contre OFFSET à 1 000 000 de commandes : 944,3 → 0,177 ms, identique à petite profondeur	PRINCIPAL p. 19-20	principal
Pooling	Atelier 7 : PgBouncer 40 → 5 connexions mais 261 → 94 transactions/s ; Jour 5 : pool 5 / 10 / 20	PRINCIPAL p. 21 ; P-06, P-07	variante rejetée
Cache	Atelier 8 : miss 2,7 ms, hit 0,988 ms ; 57.50 servi pour 19.90 ; invalidation, expiration, repli, course	PRINCIPAL p. 22-24	fraîcheur
Charge	Jour 5 : courbe 1/5/10/20 clients (N+1 ×4,2), charge ouverte (503 à 130 %), rapport concurrent p95 +64 %	atelier9/ ; P-05, P-12	principal
Donnée moderne	Jour 5 : export journalier CSV (240 lignes, 80 000 commandes payées, montant exact), contrat de données ; Lakehouse et Data Mesh conçus, non déployés	atelier9/export	contexte
Non réalisé	Réplication, partitionnement, sharding : aucune preuve produite	—	absent

Choix. Une optimisation forte plutôt que plusieurs gains isolés : le chargement groupé de GET /commandes (problème → mécanisme → mesure → limite complets), avec le cache Redis comme preuve de fraîcheur. Les autres familles sont des preuves de contexte ; elles ne sont pas redéfinies ici.

A. Besoin et état initial

Élément	Contenu	Preuve
Endpoint	GET /commandes?limit=20 : écran « mes commandes » du client 42, 20 commandes avec leurs lignes (3 lignes par commande)	P-03
Volume	100 000 commandes, 1 000 clients ; client 42 : 100 commandes (5 pages de 20)	P-03, P-14
Charge	1, 5, 10 et 20 clients simultanés (charge fermée), 10 s de mesure après 3 s d'échauffement, 3 essais alternés	P-05
Problème observable	À 20 clients : 246,9 req/s , p95 98,0 ms , file du pool jusqu'à 15 requêtes, CPU ≈ 50 %, 0 erreur. Le débit plafonne dès 5 clients (239 → 251 → 247 req/s) : le service est saturé sans erreur.	P-01
Métrique de référence	Débit terminé et p95 de GET /commandes à 20 clients, N+1, pool de 5 connexions : 246,9 req/s ; 98,0 ms	P-01

B. Architecture et stockage



Composant réellement utilisé	Détail	Preuve
PostgreSQL 18.6	laboratoire api-postgres-1 : schéma shopflow (100 000 commandes, 300 000 lignes, 6 index issus des contraintes). L'index composé de l'atelier 3 a été testé puis supprimé, sa migration est écrite mais non appliquée au laboratoire ; la migration progressive de l'atelier 4 bis a été rejouée sur une copie	P-13
Redis 8.10.2	cache-aside de la fiche produit : miss = 1 SELECT, puis copie avec TTL 60 s ; l'écriture (PATCH) supprime la clé ; politique allkeys-lru	P-08
Pool de l'API	POOL_MAX = 5 sauf série pool ; abandon après 1 s d'attente (connectionTimeoutMillis = 1000, réponse 503)	P-06, P-12
Hors périmètre	réplication, partitionnement, sharding : non réalisés ; Lakehouse et Data Mesh : conçus, non déployés	—

C. Intervention principale : un seul changement, 21 requêtes → 2

Question de la fiche	Réponse
Hypothèse causale	Le plafond vient du nombre d'allers-retours (une requête de lignes par commande : $1 + N = 21$), pas de la requête SQL ni de la taille du pool.
Changement (un seul)	relations=n1 → relations=groupe : une requête de lignes pour toute la page, WHERE commande_id = ANY(\$1::bigint[]); ni index, ni pool, ni base modifiés.

Avant (N+1)	Après (groupé)
<pre>rows = SELECT ... commandes LIMIT 20; POUR CHAQUE commande : SELECT ... FROM lignes WHERE commande_id = \$1; -- 1 + 20 = 21 requêtes</pre>	<pre>rows = SELECT ... commandes LIMIT 20; SELECT ... FROM lignes WHERE commande_id = ANY(\$1::bigint[]); -- 2 requêtes, toujours</pre>

Trace montrant le mécanisme (P-02, P-03)

Observation	N+1	Groupé	Lecture
Requêtes SQL par page de 20 / 50 / 100 commandes	21 / 51 / 101	2 / 2 / 2	croît avec N, contre constant
pg_stat_statements : appels de la requête de lignes (1 200 pages)	20 000 × 0,018 ms	200 × 0,186 ms	le signal est le nombre d'appels
Journal des requêtes lentes (seuil 20 ms)	requête de lignes absente	absente	0 ligne sur 1249
Temps dans l'API (sqlMs, page de 20)	14,2 ms (≈ 0,68 ms × 21)	2,2 ms	×38 le temps vu par PostgreSQL (0,018 ms)

vu par PostgreSQL (1 requête)

0,018 ms

vu par l'API (1 requête, aller-retour)

0,68 ms

Suspects écartés

Suspect	Verdict	Preuve
La requête SQL	innocente	0,018 ms vue par PostgreSQL ; absente du journal lent ; non classée par temps cumulé (P-02)
Le pool de connexions	conséquence	file jusqu'à 15 ; pool de 20 : 276 à 299 req/s, loin de 1032 (P-06)
Les allers-retours	cause retenue	21 par page, chacun environ 38 fois plus cher côté API que côté PostgreSQL (P-02)

Limite du mécanisme. Sur une page isolée le gain de durée est modeste (HTTP 55,3 → 43,9 ms, base locale, aucun autre client) : il devient grand sous concurrence, parce que chaque page occupe une connexion du pool pendant 21 allers-retours (P-03, P-05).

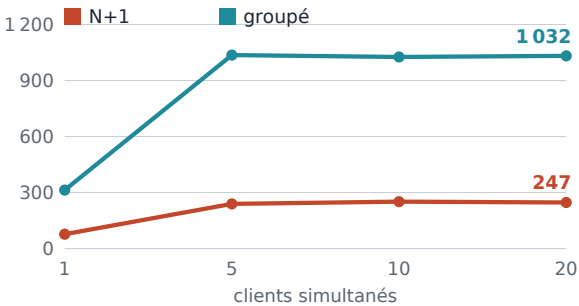
D. Benchmark avant / après (même protocole, un seul changement)

Protocole. Charge fermée sur GET /commandes?limit=20 (mêmes 20 commandes), 1/5/10/20 clients, **10 s de mesure après 3 s d'échauffement**, 3 essais en ordre alterné (N+1, groupé, N+1...), médiane retenue ; API, PostgreSQL et générateur sur la même machine (i5-10310U, 8 cœurs, 7,6 Go, WSL2). Fichiers : atelier9/resultats/p2_charge.json et p2_run.log (P-05).

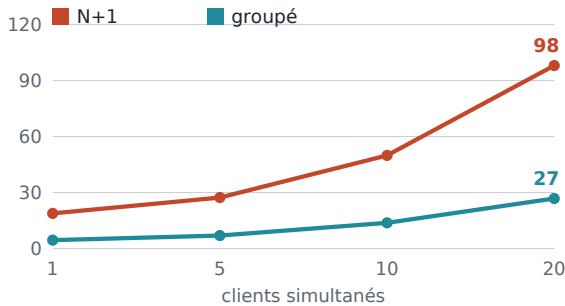
Clients	N+1 req/s	p50 ms	p95 ms	Err.	Groupé req/s	p50 ms	p95 ms	Err.
1	77,0	12,0	18,8	0	313,2	3,0	4,5	0
5	239,4	19,8	27,3	0	1036,1	4,5	7,0	0
10	251,3	38,1	49,9	0	1026,1	9,0	13,8	0
20	246,9	79,0	98,0	0	1032,1	18,2	26,8	0

Médiane de 3 essais par ligne ; erreurs = réponses non 200 (0 partout).

Requêtes terminées par seconde



Latence p95 (ms)



Lecture de la saturation et reproductibilité

Constat à 20 clients	N+1	Groupé	Effet
Débit médian	246,9 req/s	1032,1 req/s	×4,2
p95 médian	98,0 ms	26,8 ms	−73 %
3 essais (req/s)	238 à 253	978 à 1 082	intervalles disjoints : reproductible
CPU moyen de la machine / erreurs	50,2 % / 0	50,1 % / 0	le gain vient du travail évité, pas d'une ressource en plus

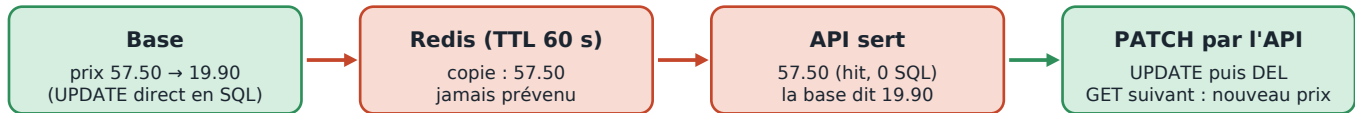
Saturation. Le N+1 plafonne dès 5 clients : les clients supplémentaires n'ajoutent que de l'attente (file du pool jusqu'à 15). Le groupé plafonne plus haut (≈ 1 032 req/s) avec 5 connexions seulement. La charge fermée masque l'effondrement : en charge ouverte à 90 % du débit fermé le comportement est instable (un essai p95 359 ms, l'autre 503), à 130 % presque tout échoue en 503 (P-12) ; ces essais sont présentés un par un, sans médiane.

E. Correction et fiabilité

1. Invariant fonctionnel : le résultat est identique (P-03, P-04)

Contrôle	Obtenu	Attendu
JSON complet (commandes et lignes) N+1 = groupé, pages de 5 / 20 / 50 / 100	identique ; 6 / 21 / 51 / 101 SQL contre 2	même contenu
Sans jeton / client_id injecté dans l'URL / curseur falsifié	401 / 400 / 400	refus
Laboratoire après les tests (P-14)	6 index, 0 statistique étendue, md5 des commandes inchangé	inchangé

2. Fraîcheur du cache Redis : ce qu'il gagne, ce qu'il peut faire mentir (P-08 à P-11)



Scénario	Résultat mesuré	Ce que ça établit
Miss puis hit (100 paires, connexion persistante)	miss : 1 SELECT, 2,7 ms ; hit : 0 SQL, 0,988 ms (x2,7)	gain petit (base locale, clé primaire) ; le vrai gain est la charge évitée
UPDATE direct en SQL, puis lecture API	l'API sert 57.50 (hit, 0 SQL), la base a 19.90 ; après DEL : 19.90	le TTL n'actualise pas la copie : elle reste périmée jusqu'au DEL ou à l'expiration
PATCH par l'API (29.90), puis 2 lectures	invalidation = ok ; 1er GET : 29.90 (miss) ; 2e GET : hit, 0 SQL	le chemin d'écriture prévu invalide la copie
Expiration	TTL 60 → -2 après 61 s ; relecture : miss, 1 SELECT	le TTL borne la copie périmée à 60 s
Redis arrêté	200, Cache = indisponible, 1 SELECT par lecture	repli correct, mais toute la charge revient à PostgreSQL
PATCH avec Redis arrêté	invalidation = échouée ; base 34.90 ; copie 29.90 servie au retour de Redis	l'écriture est valide, la copie reste ancienne
Course lecture lente / invalidation (à la main)	A lit 34.90 ; B écrit 39.90 puis supprime la clé ; A range 34.90 : l'API sert 34.90, la base a 39.90	supprimer la clé ne supprime pas toutes les courses (pistes : version, invalidation rejouée)
Rafale : 20 lectures simultanées, clé absente	6 miss, 6 SELECT au lieu d'un ; clé présente : 20 hit, 0 SQL	une mesure, non répétée ; chiffre non reproductible à l'unité près

Garantie retenue. Redis n'est pas la source de vérité : fraîcheur acceptée de quelques secondes à 60 s pour la fiche catalogue ; la validation d'un achat lit prix et stock dans PostgreSQL, jamais dans le cache.

F. Décision et limites

	Contenu	Preuve
Retenu	Chargement groupé : 21 → 2 requêtes, débit ×4,2, p95 –73 %, 0 erreur, réponse identique. Cache de la fiche produit avec TTL 60 s et invalidation par PATCH.	P-02 à P-05, P-08
Variante rejetée n° 1 agrandir le pool 5 → 20	supprime l'attente (0 au lieu de 15) mais seulement 276 à 299 req/s contre 1 032 avec le groupé et 5 connexions ; 2 essais, indicatif	P-06
Variante rejetée n° 2 PgBouncer (mode transaction)	connexions 40 → 5 mais débit 260,7 → 93,8 transactions/s, latence 153 → 422 ms : il borne les connexions, il n'accélère rien	P-07
Coûts	groupé : aucun index ni écriture en plus ; liste de 100 identifiants au plus par requête (limite de page de l'API) ; regroupement des lignes en mémoire. Cache : copie périmée jusqu'à 60 s, plus de SELECT si Redis tombe.	P-03, P-09, P-10
Risques restants	course entre lecture lente et invalidation (reproduite à la main, non corrigée) ; rafale de rechargements sur clé absente ; invalidation échouée = copie ancienne jusqu'au TTL	P-10, P-11
Tests encore nécessaires	plusieurs instances d'API ; réseau réel (le gain devrait croître avec la latence, non mesuré) ; plus de 20 clients ; mesures de 30 s ; production	—

Conclusion testable

Valable ici : cette machine (8 cœurs, 7,6 Go, WSL2), ce jeu (100 000 commandes, 1 000 clients), GET /commandes?limit=20, 1 à 20 clients, 10 s après 3 s d'échauffement. **Non établi** : un gain en production ; le comportement avec plusieurs instances d'API ; que l'index composé (atelier 3) ou le cache accélèrent « tout PostgreSQL ». La fiche cite 10 s d'échauffement et 30 s de mesure comme exemple : mon protocole est plus court, l'écart entre séries (×4) dépasse l'écart entre essais d'une même série.

Fiches d'identité des preuves

Chaque fiche donne : identifiant · question · conditions et action · résultat brut (fichier, annexe) · interprétation · limite.

ID · Question	Conditions · Action	Résultat brut (où)	Interprétation	Limite
P-01 ÉTAT-INITIAL Quel problème observe-t-on sur GET /commandes ?	N+1, pool 5, 20 clients fermés, 10 s après 3 s, médiane de 3 ; p2_charge.py série B	p2_charge.json (série B), p2_run.log	246,9 req/s, p95 98,0 ms, file du pool 15, 0 erreur : saturé sans erreur	charge fermée, une machine
P-02 PGSS-N1 Le coût du N+1 est-il dans le SQL ?	copie jetable a9-pg (PostgreSQL 18.6, md5 identiques au laboratoire), pg_stat_statements ; 1 000 pages N+1 + 200 groupées	p1_diagnostics.json, journal de la copie	20 000 appels × 0,018 ms, absente du journal lent ; API : 14,2 ms pour 21 requêtes (×38 par requête)	le signal est le nombre d'appels, pas la durée
P-03 COMPTAGE-ÉQUIV Le groupé donne-t-il le même résultat avec moins de requêtes ?	client 42, 100 commandes ; mêmes commandes en relations=n1 puis groupe (run_partie_a.py)	resultats_partie_a.json, api_journal.log (Traceld) ; PRINCIPAL p. 19-20, 53-56	6/21/51/101 SQL contre 2 ; JSON identique ; HTTP 55,3 → 43,9 ms (page isolée)	base locale : gain de durée modeste hors concurrence
P-04 DROITS Les droits sont-ils conservés ?	appels sans jeton, avec client_id dans l'URL, avec curseur falsifié	resultats_partie_a.json (5_droits) ; PRINCIPAL p. 19	401 / 400 / 400	testé sur cet endpoint seulement
P-05 BENCH-COURBE Le gain tient-il quand la concurrence augmente ?	1/5/10/20 clients, 10 s après 3 s, 3 essais alternés, un seul changement ; p2_charge.py	p2_charge.json (essais bruts), p2_run.log	N+1 ≈ 250 req/s dès 5 clients ; groupé ≈ 1 032 ; à 20 clients ×4,2, p95 –73 %, essais disjoints (238 à 253 contre 978 à 1 082)	générateur et API sur la même machine ; 10 s (fiche : 30 s en exemple)
P-06 POOL Agrandir le pool suffit-il ?	N+1, 20 clients, POOL_MAX 5 / 10 / 20, 2 essais par taille	p2_charge.json (C)	pool 20 : attente 0, 276 à 299 req/s contre 137 à 216 à pool 5 ; loin de 1 032	2 essais, pool 5 instable : indicatif
P-07 PGBOUNCER PgBouncer accélère-t-il ?	laboratoire séparé, 5 connexions serveur, mode transaction, 40 clients persistants, médiane de 3	atelier7/resultats/pgbouncer, b_compare40.log ; PRINCIPAL p. 21, 53-56	connexions 40 → 5 ; 260,7 → 93,8 tps ; latence 153 → 422 ms	campagnes de 20 s ; test à 10 clients instable (ordre inversé)
P-08 CACHE-HITMISS Combien de travail le cache évite-t-il ?	produit 42, clé shopflow:produit:v1:42, TTL 60 s, 100 paires miss/hit, connexion persistante	mesure_persistante.json ; PRINCIPAL p. 23, 57-58	miss : 1 SELECT, 2,7 ms ; hit : 0 SQL, 0,988 ms (×2,7)	gain absolu petit : base locale, accès par clé primaire
P-09 CACHE-STALE Que sert l'API après une modification SQL directe ?	UPDATE prix = 19.90 en SQL sans passer par l'API, puis lecture ; run_atelier8.py	resultats_atelier8.json, api_extraits_par_traceid.txt ; PRINCIPAL p. 22-23, 58	l'API sert 57.50 (hit, 0 SQL) alors que la base a 19.90 ; après DEL : 19.90	un produit, un processus API
P-10 CACHE-INVALID Invalidation, expiration et repli fonctionnent-ils ?	PATCH 29.90 ; attente 61 s ; Redis arrêté ; PATCH 34.90 avec Redis arrêté	resultats_atelier8.json ; PRINCIPAL p. 22-23, 58	invalidation ok puis miss ; TTL 60 → –2 ; Redis arrêté : 200 et 1 SELECT ; invalidation échouée : base 34.90, copie 29.90	seul TTL testé : 60 s
P-11 CACHE-COURSE Supprimer la clé après l'écriture suffit-il ?	course lecture lente / invalidation reproduite à la main ; rafale de 20 lectures (stampede.mjs)	resultats_atelier8.json ; PRINCIPAL p. 23, 59	course : l'API sert 34.90, la base a 39.90 ; rafale : 6 miss et 6 SELECT	course non reproduite sous charge ; rafale non répétée
P-12 CHARGE-OUVERTE La charge fermée masque-t-elle la saturation ?	N+1, pool 5, arrivées à cadence fixe : 50 %, 90 %, 130 % du débit fermé, 2 essais	p2_charge.json (D)	50 % : tient ; 90 % : un essai p95 359 ms, l'autre 503 ; 130 % : presque tout en 503 (pool 1 s)	comportement bimodal : pas de médiane
P-13 STOCKAGE Quels éléments de gestion de la donnée sont réellement déployés ?	ateliers 3, 4 bis, 8 : DDL, index testés, migration progressive, Redis ; état du laboratoire vérifié avant et après chaque test	atelier3/migration, atelier4bis/ ; PRINCIPAL p. 7-18, 48-52	laboratoire : 6 index de contraintes, cache Redis en service (P-08) ; index composé et migration par lots testés, non appliqués au laboratoire	pas de réplication, partitionnement ni sharding
P-14 MÉTHODE-ENV Un autre poste peut-il reproduire la mesure ?	versions, copie jetable, empreintes md5, scripts atelier9/*.py et loadgen.mjs	p1_diagnostics.json, p2_charge.json, atelier9/README.md	PostgreSQL 18.6, Node 22.14, md5 commandes 7505112bb794 et lignes e989d5bec39b identiques laboratoire / copie ; p2_charge.py : 16 min	WSL2, 8 cœurs ; journaux pgbench par transaction non conservés

Auto-évaluation B4C8 avant remise

Critère	Preuve suffisante si...	Statut	Référence
Besoin	endpoint, volume et problème définis	directe	P-01, P-03
Stockage	un élément de gestion de la donnée réellement déployé	directe (PostgreSQL, Redis en service ; index testés, non appliqués)	P-13, P-08
Correction	résultat, droits et invariants valides après l'intervention	directe	P-03, P-04
Méthode	protocole reproductible : versions, jeu, charge, durée, commandes	directe ; 10 s après 3 s, plus court que l'exemple de la fiche	P-14, P-05
Intervention	le mécanisme explique le gain	directe (appels, temps par requête)	P-02, P-03
Mesure	avant/après comparable, p95/débit/erreurs, sorties brutes	directe ; journaux pgbench non conservés (séries SQL)	P-05
Fiabilité / fraîcheur	panne, reprise, cache ou migration vérifié	directe (cache : stale, invalidation, expiration, panne, course)	P-09, P-10, P-11
Limites	coûts, risques et conditions de généralisation exposés	directe	P-06, P-07, P-12 · section F

Les deux questions

Reproductibilité : « Si je vous donne un autre poste, comment reproduisez-vous exactement votre mesure ? »

1	Créer la base avec 01_schema.sql et 02_donnees.sql (PostgreSQL 18.6) et vérifier les empreintes md5 : commandes 7505112bb794..., lignes e989d5bec39b... (P-14).
2	Démarrer l'API (Node 22.14, server.mjs, POOL_MAX = 5, port 3001) puis lancer python3 atelier9/p2_charge.py (≈ 16 min : 1/5/10/20 clients, 10 s après 3 s, 3 essais alternés).
3	Comparer l'ordre de grandeur et les intervalles, pas la valeur exacte : plafond du N+1 ≈ 250 req/s et du groupé ≈ 1 030 req/s, essais disjoints. Un autre matériel donne d'autres valeurs absolues.

Conséquence pour l'API : « Qu'est-ce que votre décision change ? »

Contrat	inchangé : même JSON, mêmes droits (401 / 400 / 400) (P-03, P-04)
Disponibilité	2 allers-retours par page au lieu de 21 : à 5 connexions le débit est ×4,2 ; Redis arrêté : l'API répond 200 en repli sur PostgreSQL (P-10)
Fraîcheur	fiche produit : copie jusqu'à 60 s, périmée après une modification SQL directe (57.50 pour 19.90) ; l'achat lit PostgreSQL (P-09)
Ressources	aucun index ni écriture en plus ; au plus 100 identifiants par requête de lignes ; en cas de repli, une lecture SQL par appel (P-10)

Checklist de remise

Vérification	OK	Où
Besoin et état initial compréhensibles sans explication orale	✓	A
Architecture et stockage réellement déployés visibles	✓	B, P-13
Chaque chiffre annoncé renvoie à une mesure réelle et à son protocole	✓	P-01 à P-14
Benchmark avant/après aux mêmes conditions sauf le changement étudié	✓	D, P-05
p95, débit et erreurs présents ; unités indiquées	✓	D
Résultat fonctionnel vérifié, pas seulement la vitesse	✓	E, P-03, P-04
Preuve de fiabilité ou de fraîcheur documentée	✓	E, P-09 à P-11
Variante rejetée et risque restant expliqués	✓	F, P-06, P-07
Conclusion limitée à la machine, au jeu et à la charge testés	✓	F
Sorties brutes, plans, traces ou Traceld conservés	✓	atelier*/ , PRINCIPAL

Formulation finale

1. Observé. À 20 clients, GET /commandes?limit=20 en N+1 plafonne à 246,9 req/s (p95 98,0 ms, file du pool jusqu'à 15, CPU 50 %, 0 erreur) pour 21 requêtes SQL par page.

2. Pourquoi. Chaque page paie 21 allers-retours, environ 0,68 ms chacun côté API contre 0,018 ms côté PostgreSQL ; le chargement groupé les ramène à 2 sans changer la réponse, d'où 1032,1 req/s et un p95 de 26,8 ms (3 essais disjoints). Aggrandir le pool ou ajouter PgBouncer ne produit pas cet effet.

3. Conditions. Je recommande le chargement groupé sur cette machine, ce jeu et 1 à 20 clients ; le cache de la fiche produit est acceptable avec un TTL de 60 s car il peut servir une valeur périmée (57.50 pour 19.90) ; il reste à tester plusieurs instances d'API, un réseau réel, des mesures de 30 s et la production.